LELEC2531 Project

Context

This project aims to create and integrate a hardware accelerator into the DE0 Nano and Raspberry Pi architecture, as illustrated in Figure 1. The project is divided into two parts:

1. **Accelerator design (Weeks 3-6):** Create the hardware accelerator module to perform the desired operation. It should be a sequential circuit containing input and output registers .
2. **Integration (Weeks 9-11):** Integrate the hardware accelerator module with a pipelined processor within the DE0-Nano board, and communicate with it using the Raspberry Pi board.
3. **Optimization (Weeks 12-13):** Optimize the hardware accelerator module and/or its integration for improved efficiency or extended configurations.

If parts 1 and 2 are properly implemented, this guarantees that the project meets **the minimal criteria to pass the course**. Then, any improvement on top of this baseline will increase your grade, so these improvements should always be compared to the baseline in your presentation and report.

The project is designed to give you practical insights about the entire course stack, with a focus on: **(1) multi-level hardware** design, **(2) pipelined processor** architecture, **(3)** the various **abstraction levels** covered in the course, **(4)** system-level **bus transfers**.

1 Adding a dedicated accelerator

Why do we need an accelerator ?

Let's consider a scenario where we want the pipelined processor to perform **computations that are not natively supported by its instruction set**. What options does the designer have? The most straightforward approach would be to write a **program in assembly language**. However, since the instruction set does not include a dedicated instruction to perform specialised tasks, you would need to break down complex mathematical operations into a series of basic instructions such as additions, subtractions, and multiplications. While this method will eventually produce the correct result, it requires writing a **complex assembly program** and consumes **a significant number of processor cycles** to complete.

Now imagine needing to make numerous computation-intensive tasks. The total number of processor cycles required would scale proportionally, leading to **excessive computing time and power consumption**. Moreover, while the processor is engaged in these operations, **it cannot perform other tasks**, leading to a **bottleneck**. Clearly, a more efficient solution is needed.

You might consider **changing the processor's architecture** to include a custom instruction. Although this approach is **theoretically possible**, it poses a **significant issue**. In real-world situations, processors are typically licensed as **commercial intellectual properties** (IPs) by companies such as Intel, AMD, or ARM. Thus, these designs are provided as closed systems, or "black boxes" that do not permit modifications to their internal architecture.

To cope with these issues, we can directly design a specialized hardware block to perform this operation. This hardware block is called "accelerator". Doing your own accelerator has the following advantages:

1. **Performance improvement:** It executes computations in **fewer cycles** than a general-purpose processor.
2. **Parallel operation:** The accelerator manages tasks while the **processor remains available** for other operations.
3. **No processor modification:** The processor's **architecture stays unchanged**, avoiding complexity and licensing issues.

Accelerators are commonly used to offload demanding tasks from the processor. Examples include image processing tasks (e.g., those in graphics cards), cryptographic operations, and the encoding and decoding of compressed files, among many other compute-intensive applications. This approach not only improves efficiency but also illustrates the versatility of hardware-software co-design in modern systems.

Specific to your project

Part 1: Accelerator design

In this part, you will have to design the hardware accelerator performing the function explained in the project topic description. Your **starting point** should be to understand well the topic with the suggested references and start working on a first functional version of the accelerator architecture. The minimum requirements for this hardware accelerator are:

1. **Input Register:** One 32-bit register to hold the input data for computations.
2. **Configuration Registers:** Two 8-bit registers to modify and control the behavior of the hardware function implemented by the accelerator.
3. **Output Register:** One 32-bit register to store and provide the computation results.

Deliverables At the end of the first part, we will ask you to write an intermediary report for the project, focused on the hardware accelerator design.

Part 2: Accelerator integration

Before beginning this part of the project, it is essential to have a proper understanding of system-level data transfers, particularly memory access. We strongly recommend reviewing the material from **Labs 4 and 5**, as well as the architecture illustrated in Figure 1. To ensure you have a thorough grasp of Figure 1, you should be able to answer the following questions:

1. What is the purpose of each SystemVerilog .sv file?
2. How does the pipelined processor determine where to read or write data?
3. Where are the memory addresses defined?

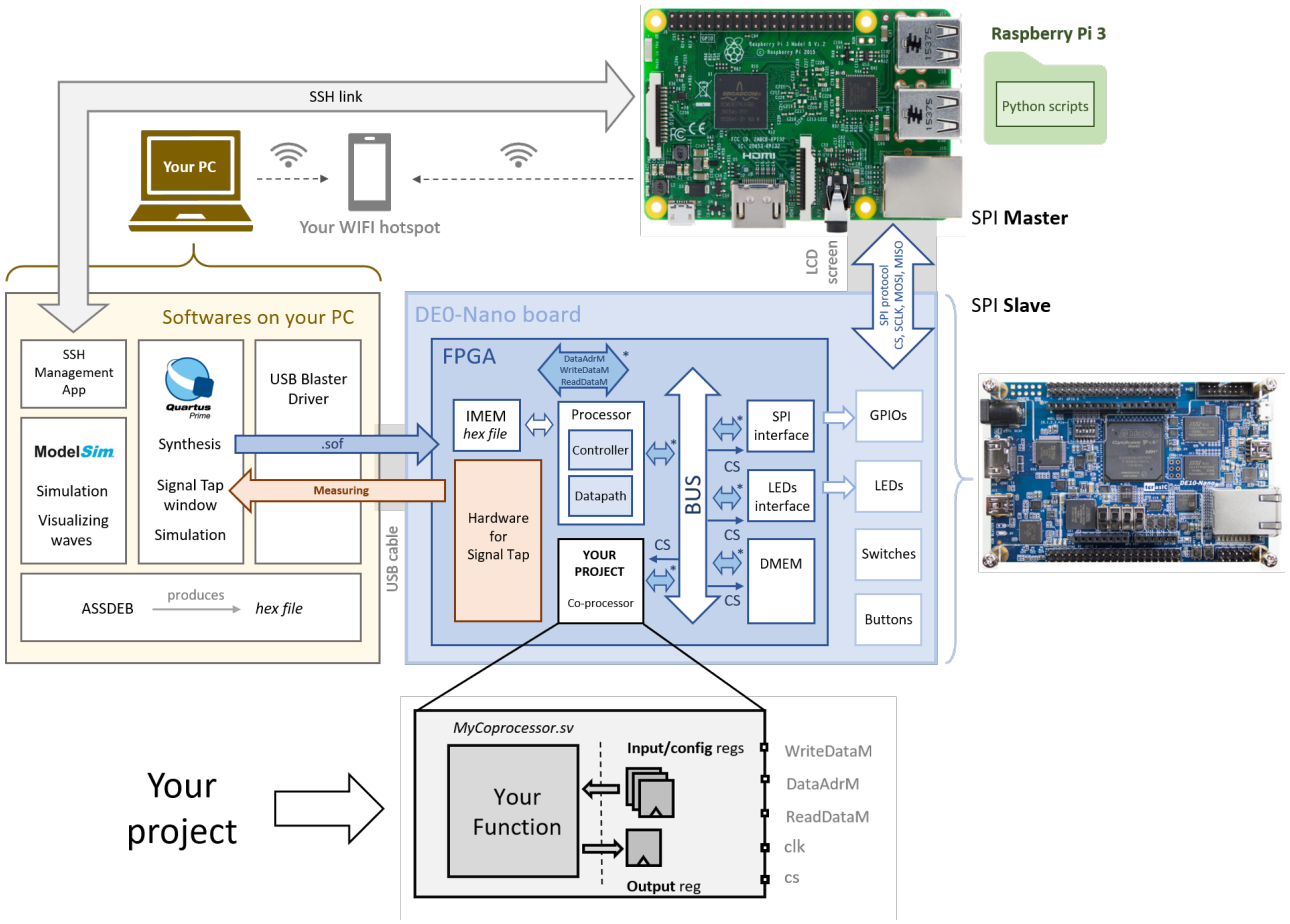


Figure 1: Full LELEC2531 setup including Raspberry Pi and DE0 nano boards, highlighting your project.

The starting point of this second part will be the folder containing the pipelined processor architecture introduced in **Lab 5**. You will **integrate the previously built accelerator** into the **system bus** by assigning it a specific **memory-mapped address range**. Accessing the accelerator (whether to read, write, or trigger computations) will operate under the **same principles as peripheral access** (e.g., the SPI unit). During the MEM stage of memory instruction execution, if the target address falls within the accelerator's range, the pipelined processor will interact with its interface registers instead of the DMEM. This mechanism allows you to **send input data** to the accelerator, **retrieve its output**, and **configure its behavior** by writing to its configuration registers.

Once your hardware accelerator is functional and integrated, we ask you quantify its performance. The performance metrics that we ask you to use are: (1) FPGA hardware usage (number of logic, LUT, memory blocks used), (2) area occupancy and (3) its estimated power consumption.

Deliverables (optional) During this second part, you will have the opportunity to submit a status report to showcase the project progress so far. This report will be read by the teaching staff and you will get feedback on it the following week.

Part 3: Accelerator optimization

During this third part, you will upgrade/expand your initial hardware accelerator from Part 1. This can be done by either upgrading the accelerator performance given a specific metric (latency, throughput, hardware usage...), or by extending its capabilities (new configurations, operations...). Examples of improvements include building a larger accelerator, improve the throughput/latency of the system, do a direct memory access (DMA), perform Single Instruction Multiple Data architecture (SIMD), etc.

Again, we ask you to quantify the performance metrics of your accelerator (hardware usage, area, power consumption), and compare it to the baseline of the initial integrated accelerator in part 2.

Deliverables The end of the third part marks the end of the project. We will ask you to write a final project report as well as showcase your work during an oral presentation.

2 The day of the examination

You are expected to:

1. Demonstrate the accelerator operation:
   - Send the **two configuration registers** and the **32-bit input data** to the accelerator **via SPI** by executing a **Python script** on your **Raspberry Pi**.
   - Retrieve the **output data** from the accelerator **on your Raspberry Pi** via **SPI**.
   - Confirm that the SPI transfer occurred by setting the appropriate **triggering condition in SignalTap**.
2. Assess your accelerator implementation:
   - **Review your code** with us during the presentation.
   - **Discuss the advantages** of integrating your accelerator into the overall system architecture. What is its performance in terms of the following KPI : (1) FPGA hardware usage (number of logic, LUT, memory blocks used), (2) area, and (3) the estimated power consumption. Compare the performance of a CPU implementation (no accelerator, only pipelined processor) to your initial accelerator implementation (parts 1-2), and to the optimized one (parts 3). If specific performances are difficult to evaluate, please specify why.